

THE REAL-WORLD STORY

Imagine your computer filesystem: Root folder '/' contains subfolders 'Documents/' and 'Pictures/', which contain files.

THE TREE AHA MOMENT

A Tree is a directed, acyclic graph where every node has EXACTLY 1 parent (except the root, which has 0 parents) and children split into branches!

WHY TREES VS LINKED LISTS?

- Linked List:** Linear sequence (1 next pointer). Search takes $O(N)$ time.
- Binary Search Tree (BST):** Branching choice (left < root < right). Search takes **$O(\log N)$ time!**
- Hierarchy:** Expresses parent-child relationships naturally (DOM, Corporate orgs, Filesystem).

VISUALIZING BINARY SEARCH TREE

BST Invariant (Left < Root < Right):

```
      4
     / \
    2   6
   /\  /\
  1 3 5 7
```

Inorder Traversal: [1, 2, 3, 4, 5, 6, 7] (STRICTLY SORTED!)

PREREQUISITES & COMPLEXITY REASONING

Traversal / Algorithm	Time Complexity	Auxiliary Space
Tree DFS (Preorder / Inorder / Postorder)	$O(N)$ single pass	$O(H)$ call stack
Tree BFS Level Order Traversal	$O(N)$ single pass	$O(W)$ max width queue
BST Search / Insert / Delete	$O(H) = O(\log N)$ balanced	$O(H)$ call stack
Lowest Common Ancestor (LCA)	$O(N)$ single pass	$O(H)$ call stack

CLUES THAT SCREAM TREES

- "Need parent answer from children answers (height, diameter, path sum)" -> Post-order Bottom-Up DFS
- "Search elements in sorted order in BST" -> In-order DFS ('Left -> Parent -> Right')
- "Level-by-level processing / Print tree by row" -> BFS Queue Level Order Traversal

PATTERN
1

MAXIMUM DEPTH OF BINARY TREE (Bottom-Up Postorder)

e.g. Maximum Depth of Binary Tree (LC 104)

RECURRENCE RELATION

``depth(root) = 1 + max(depth(left), depth(right))``.
Base case: ``depth(null) = 0``.

TEMPLATE — LC 104

```
public int maxDepth(TreeNode root) {  
    if (root == null) return 0;  
    int leftDepth = maxDepth(root.left);  
    int rightDepth = maxDepth(root.right);  
    return 1 + Math.max(leftDepth, rightDepth);  
}
```

PATTERN 2 INVERT BINARY TREE (Preorder Swapping) e.g. Invert Binary Tree (LC 226)

SWAP LEFT & RIGHT

Swap `root.left` and `root.right` pointers at every node, then recurse down left and right children!

TEMPLATE — LC 226

```
public TreeNode invertTree(TreeNode root) {  
    if (root == null) return null;  
    TreeNode temp = root.left;  
    root.left = root.right;  
    root.right = temp;  
    invertTree(root.left);  
    invertTree(root.right);  
    return root;  
}
```

PATTERN 5

SAME TREE (Dual Node Recursion)

e.g. Same Tree (LC 100)

STRUCTURAL EQUALITY

Check `p.val == q.val`, then recurse
`isSameTree(p.left, q.left)` and
`isSameTree(p.right, q.right)`.

TEMPLATE — LC 100

```
public boolean isSameTree(TreeNode p, TreeNode q) {  
    if (p == null && q == null) return true;  
    if (p == null || q == null) return false;  
    if (p.val != q.val) return false;  
    return isSameTree(p.left, q.left) && isSameTree(p.right, q.righ  
t);  
}
```

PATTERN 6	LOWEST COMMON ANCESTOR (LCA in Binary Tree) e.g. Lowest Common Ancestor of a Binary Tree (LC 236)
SPLIT POINTER CONDITION If left search returns non-null AND right search returns non-null, current node is the LCA!	TEMPLATE — LC 236 <pre>public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) { if (root == null root == p root == q) return root; TreeNode left = lowestCommonAncestor(root.left, p, q); TreeNode right = lowestCommonAncestor(root.right, p, q); if (left != null && right != null) return root; // Split point is LCA! return (left != null) ? left : right; }</pre>

TREES & BST

MASTERCLASS

Decision Tree & Trigger Words

COMPLETE TREES & BST PROBLEM LADDER SUMMARY					
LC #	Problem	Pattern	Time	Space	Diff
226	Invert Binary Tree	Preorder Pointer Swapping	O(N)	O(H)	E
104	Maximum Depth of Binary Tree	Postorder Bottom-Up DFS	O(N)	O(H)	E
543	Diameter of Binary Tree	Height Pass Global Max Tracking	O(N)	O(H)	E
236	Lowest Common Ancestor	Split Pointer LCA Search	O(N)	O(H)	M
98	Validate Binary Search Tree	Min/Max Range Pass	O(N)	O(H)	M
230	Kth Smallest Element in BST	Inorder Traversal Counter	O(N)	O(H)	M
124	Binary Tree Max Path Sum	Postorder Path Sum Tracking	O(N)	O(H)	H

LC 226 · Invert Binary Tree

EASY

Pattern: Preorder Swap
Key Insight: Swap left & right pointers.

```
public TreeNode invertTree(TreeNode root) {
    if (root == null) return null;
    TreeNode tmp = root.left; root.left = root.right; root.right = tmp;
    invertTree(root.left); invertTree(root.right);
    return root;
}
```

LC 104 · Max Depth of Binary Tree

EASY

Pattern: Postorder Bottom-Up
Key Insight: `1 + max(left, right)`.

```
public int maxDepth(TreeNode root) {
    if (root == null) return 0;
    return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
}
```

LC 543 · Diameter of Binary Tree

EASY

Pattern: Height Pass Max Track
Key Insight: Update `left + right` max.

```
int maxD = 0;
public int diameterOfBinaryTree(TreeNode root) {
    h(root); return maxD;
}
private int h(TreeNode root) {
    if (root == null) return 0;
    int l = h(root.left), r = h(root.right);
    maxD = Math.max(maxD, l + r);
    return 1 + Math.max(l, r);
}
```


LC 236 · Lowest Common Ancestor

MEDIUM

Pattern: Split Pointer Check

Key Insight: Return root if `left != null && right != null`.

```
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
    if (root == null || root == p || root == q) return root;
    TreeNode l = lowestCommonAncestor(root.left, p, q);
    TreeNode r = lowestCommonAncestor(root.right, p, q);
    if (l != null && r != null) return root;
    return (l != null) ? l : r;
}
```

LC 98 · Validate Binary Search Tree

MEDIUM

Pattern: Min/Max Range Pass

Key Insight: Check `min < val < max`.

```
public boolean isValidBST(TreeNode root) { return v(root, null, null); }
private boolean v(TreeNode n, Integer min, Integer max) {
    if (n == null) return true;
    if ((min != null && n.val <= min) || (max != null && n.val >= max)) return false;
    return v(n.left, min, n.val) && v(n.right, n.val, max);
}
```

LC 124 · Binary Tree Max Path Sum

HARD

Pattern: Postorder Path Sum

Key Insight: `max(0, left) + max(0, right) + node.val`.

```
int maxSum = Integer.MIN_VALUE;
public int maxPathSum(TreeNode root) {
    gain(root); return maxSum;
}
private int gain(TreeNode node) {
    if (node == null) return 0;
    int l = Math.max(0, gain(node.left));
    int r = Math.max(0, gain(node.right));
    maxSum = Math.max(maxSum, 1 + r + node.val);
    return node.val + Math.max(l, r);
}
```

LC 230 · Kth Smallest Element in BST

MEDIUM

Pattern: Inorder Traversal

Key Insight: Inorder visit count == k.

```
int c = 0, res = -1;
public int kthSmallest(TreeNode root, int k) {
    in(root, k); return res;
}
private void in(TreeNode n, int k) {
    if (n == null) return;
    in(n.left, k);
    if (++c == k) { res = n.val; return; }
    in(n.right, k);
}
```

🔍 DRY RUN — Validate BST ([5, 1, 4, null, null, 3, 6])

Node	Min Limit	Max Limit	Valid Check (min < val < max)	Action
5	null	null	'null < 5 < null' ✓	Recurse left (null, 5), right (5, null)
1	null	5	'null < 1 < 5' ✓	Valid left leaf!
4	5	null	'5 < 4 < null' ✗ FALSE!	4 is right child of 5, but value 4 < 5! Return false!

📐 MATHEMATICAL PROOF — INORDER TRAVERSAL OF BST IS STRICTLY SORTED

Claim: Inorder traversal of a BST produces a strictly ascending sequence.

Proof: By BST invariant, for every node X, all left subtree nodes < X, and all right subtree nodes > X. Inorder visits 'Left -> Node -> Right'. By induction, elements are processed in strictly non-decreasing order -> **Exact Mathematical Guarantee!**

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

☐ Write Preorder, Inorder, & Postorder DFS in 2 minutes

☐ Code Invert Binary Tree with pointer swapping

☐ Calculate Max Depth & Diameter in O(N) single pass

☐ Code Lowest Common Ancestor (LCA) in binary tree

☐ Validate BST using min/max range constraints

☐ Find Kth smallest in BST via Inorder traversal

☐ Code Binary Tree Maximum Path Sum (LC 124)

☐ Explain why Inorder traversal of BST is sorted

👛 GOLDEN RULES — NEVER FORGET

Rule 1 — Use Postorder when Children's Answers are Required
When a node needs answers from its left and right subtrees (e.g. height, diameter, balanced check, max path sum), ALWAYS use Postorder Bottom-Up DFS!

Rule 2 — Guard Against Negative Path Sums
In Binary Tree Maximum Path Sum, use 'Math.max(0, gain(child))' to ignore subtrees that contribute negative path values!